

Exact Triclinic Cell-List Backend Specification

Stage S1 Neighbor Candidate Generation for mdstats

mdstats

2026-07-14

Contents

1 Purpose and implementation status	2
2 Scope	2
2.1 Included	2
2.2 Excluded	3
3 Algorithmic provenance and attribution boundary	3
4 Scientific result contract	3
5 Coordinate and lattice conventions	4
5.1 Row-vector convention	4
5.2 Search-basis transformation	4
6 Search-basis preparation	4
6.1 Optional Minkowski reduction	4
6.2 Why reduction is useful	5
7 Perpendicular cell-plane heights	5
8 Fractional bin construction	6
8.1 Periodic axes	6
8.2 Nonperiodic axes	6
8.3 Assignment	6
9 Metric-aware bin stencil	7
9.1 Bin-offset displacement region	7
9.2 Cell metric	7
9.3 Exact box minimization	7
9.4 Finite offset ranges	8
10 Candidate-bin traversal	8
11 Original-basis final geometry	9
12 Implemented internal API	9
12.1 Backend selection	9
12.2 Cell-list options	9
12.3 Shared facade	9
12.4 Internal plan and diagnostics	10
13 Complexity	10

14 Exactness argument	11
15 Error handling	11
16 Determinism	11
17 Dense-equivalence acceptance matrix	12
18 Benchmark interpretation	12
19 Interaction with consumer modules	13
20 Deferred S2 contract	13
21 Reproduction checklist	13
22 References	13

1 Purpose and implementation status

This document is the normative design and implementation specification for the stage S1 exact cell-list neighbor backend in `mdstats`.

The implementation lives in:

`mdstats/analysis/_cell_list.py`

and is selected through the existing private shared-neighbor facade in:

`mdstats/analysis/_neighbors.py`

Package version:

0.14.0a1

Stage S1 is implemented. It adds an exact single-frame cell-list backend but no trajectory cache, no Verlet skin, and no automatic backend selection.

The supported explicit backend values are:

`NeighborSearchBackend.DENSE`
`NeighborSearchBackend.CELL_LIST`

The dense backend remains the authoritative numerical oracle. Every S1 cell-list acceptance test compares the complete scientific result against a fresh dense result.

2 Scope

2.1 Included

Stage S1 provides:

- optional Minkowski reduction of the internal periodic search basis;
- fractional linked-cell bins for orthogonal, triclinic, mixed-PBC, and nonperiodic geometries;
- bin counts based on perpendicular cell-plane heights rather than basis-vector lengths;
- an exact metric-aware bin-offset stencil;
- deterministic periodic and nonperiodic bin traversal;
- deterministic candidate deduplication;
- exact final minimum-image geometry in the original cell basis;
- exact preservation of the existing CSR neighbor-result contract;
- explicit dense and cell-list backend selection;
- hard complexity limits for stencil construction;

- transparent internal diagnostics for testing and benchmarking.

2.2 Excluded

Stage S1 does not provide:

- persistent candidate reuse across frames;
- a Verlet skin;
- displacement-based cache invalidation;
- deformation-aware cache validity;
- request signatures or cache sessions;
- automatic dense/cell-list crossover selection;
- one combined search over a full pair-cutoff registry;
- consumer-level backend options in RDF, coordination, bond-angle, or connectivity public APIs.

Those capabilities belong to stages S2-S4.

3 Algorithmic provenance and attribution boundary

The spatial decomposition is the classical cell-linked-list method introduced for molecular-dynamics neighbor searching by Quentrec and Brot [1]. Published pair-list methods for arbitrary periodic boxes [2], metric-tensor treatment of general parallelepiped cells [3], and closest-point periodic search [4] provide important prior art for the general-cell setting.

The optional search-basis reduction is a direct call to ASE [5]. ASE’s low-dimensional Minkowski-reduction implementation follows the algorithmic lineage of Nguyen and Stehlé [6].

The following S1 details are mdstats-specific adaptations and are not attributed to the cited papers:

- perpendicular-plane-height bin sizing as integrated into this backend;
- exact active-set minimization of the cell metric over each fractional bin-offset box;
- deterministic handling of periodic bin aliases and candidate order;
- final minimum-image vector and image-shift recovery in the original basis;
- explicit stencil-complexity guards and dense-equivalence acceptance rules.

The citations establish the historical and mathematical foundations without implying that mdstats transcribes any one published arbitrary-cell algorithm.

4 Scientific result contract

The optimized backend must return the same `NeighborListResult` scientific content as the dense oracle.

```
@dataclass(frozen=True, slots=True)
class NeighborListResult:
    frame_index: int
    center_indices: NDArray[np.int64]
    neighbor_indices: NDArray[np.int64]
    offsets: NDArray[np.int64]
    vectors: NDArray[np.float64]
    distances: NDArray[np.float64]
    image_shifts: NDArray[np.int64]
    cutoff: float
    pair_counting: PairCounting
    backend: NeighborSearchBackend
```

The following are normative:

- strict cutoff: `distance < cutoff`;
- self pairs are excluded;
- partially overlapping explicit selections are rejected;

- `UNORDERED_IDENTICAL` requires identical center and candidate selections and retains only $i < j$;
- minimum-image vectors use the original cell and PBC mask;
- image shifts use the original cell basis and satisfy

$$\mathbf{d}_{ij}^{\text{MIC}} = \mathbf{r}_j - \mathbf{r}_i + \mathbf{m}_{ij}H;$$

- candidate-selection order is preserved within each center row;
- result arrays are immutable;
- the backend label is provenance and does not alter scientific equality.

A cell-list result is accepted only when it has exact pair and image-shift identity with the dense result and tolerance-bounded equality of vectors and distances.

5 Coordinate and lattice conventions

5.1 Row-vector convention

The physical cell matrix is

$$H = \begin{pmatrix} \mathbf{a}^\top \\ \mathbf{b}^\top \\ \mathbf{c}^\top \end{pmatrix},$$

and Cartesian coordinates are

$$\mathbf{r} = \mathbf{s}H.$$

The implementation uses wrapped fractional coordinates along periodic axes and retains nonperiodic fractional coordinates unchanged.

5.2 Search-basis transformation

An internal equivalent lattice basis may be defined by an integer unimodular matrix U :

$$H_s = UH, \quad \det U = \pm 1.$$

The same Cartesian position is represented in the search basis by

$$\mathbf{s}_s = \mathbf{s}U^{-1},$$

because

$$\mathbf{s}_s H_s = \mathbf{s}U^{-1}UH = \mathbf{s}H.$$

The search basis is used only for spatial partitioning. Final pair vectors, distances, and image shifts are recomputed in the original basis.

6 Search-basis preparation

6.1 Optional Minkowski reduction

When at least two axes are periodic and `CellListOptions.use_lattice_reduction` is true, the implementation requests a Minkowski-reduced basis from ASE [5]. ASE's implementation follows the low-dimensional lattice-reduction work of Nguyen and Stehlé [6].

The returned transformation must satisfy:

- the transformation is integer-valued;
- $|\det U| = 1$;
- $H_s = UH$ within numerical tolerance;
- U^{-1} is integer-valued;
- nonperiodic basis vectors are unchanged;
- periodic vectors do not acquire contributions from nonperiodic vectors.

Any violation is an `InvalidCellGeometryError`.

Reduction is an efficiency transformation, not a scientific transformation. A reduction failure must never be hidden by silently returning a geometrically inconsistent result.

6.2 Why reduction is useful

A highly skewed basis may hide a short lattice direction in an integer combination such as

$$\mathbf{b} - \mathbf{a}.$$

A reduced basis tends to expose shorter, more nearly orthogonal periodic vectors. This increases useful bin counts and decreases the number of atom pairs found in neighboring bins.

Correctness does not depend on reduction. Dense-equivalence tests must pass with reduction enabled and disabled.

7 Perpendicular cell-plane heights

The linked-cell dimensions are based on perpendicular distances between fractional coordinate planes.

For the row-vector cell, let

$$B = H^{-1}.$$

The normal associated with fractional coordinate s_α is the column $B_{:, \alpha}$. The perpendicular height of the cell along fractional axis α is

$$h_\alpha = \frac{1}{\|B_{:, \alpha}\|}.$$

For any Cartesian displacement

$$\Delta \mathbf{r} = \Delta \mathbf{s} H,$$

we have the lower bound

$$\|\Delta \mathbf{r}\| \geq |\Delta s_\alpha| h_\alpha.$$

This is the key reason perpendicular heights are appropriate for skewed cells. A large separation in a fractional coordinate cannot be hidden by basis-vector cancellation once it is measured normal to the corresponding lattice planes.

8 Fractional bin construction

8.1 Periodic axes

For periodic axis α , the wrapped fractional domain is $[0, 1)$. Given physical cutoff r_c , the bin count is

$$n_\alpha = \max \left(1, \left\lfloor \frac{h_\alpha}{r_c} \right\rfloor \right).$$

The fractional bin width is

$$w_\alpha = \frac{1}{n_\alpha}.$$

This choice gives a perpendicular physical bin thickness

$$w_\alpha h_\alpha \geq r_c$$

except for the single-bin case, where the complete periodic direction is one bin.

8.2 Nonperiodic axes

For nonperiodic axis α , let the selected center/candidate union span

$$s_\alpha^{\min} \leq s_\alpha \leq s_\alpha^{\max},$$

with

$$L_\alpha^{(s)} = s_\alpha^{\max} - s_\alpha^{\min}.$$

The bin count is

$$n_\alpha = \max \left(1, \left\lfloor \frac{L_\alpha^{(s)} h_\alpha}{r_c} \right\rfloor \right).$$

For nonzero span,

$$w_\alpha = \frac{L_\alpha^{(s)}}{n_\alpha}.$$

If the span is numerically zero, one bin is used with a positive conservative width. Coordinates at the upper numerical boundary are clipped into the last bin.

8.3 Assignment

For bin origin o_α and width w_α , the preliminary integer index is

$$b_\alpha = \left\lfloor \frac{s_\alpha - o_\alpha}{w_\alpha} \right\rfloor.$$

Periodic indices are reduced modulo n_α . Nonperiodic indices are clipped to $[0, n_\alpha - 1]$.

9 Metric-aware bin stencil

9.1 Bin-offset displacement region

Consider one center bin and a candidate bin displaced by integer bin offset

$$\mathbf{k} = (k_1, k_2, k_3).$$

For width vector

$$\mathbf{w} = (w_1, w_2, w_3),$$

the possible fractional displacement between any point in the center bin and any point in the candidate-bin image lies in the box

$$D_{\mathbf{k}} = \prod_{\alpha=1}^3 [(k_{\alpha} - 1)w_{\alpha}, (k_{\alpha} + 1)w_{\alpha}].$$

Boundary bins can be shorter than the nominal width. Using the full nominal box is conservative and can only create extra candidates.

9.2 Cell metric

The Cartesian squared norm of fractional displacement $\mathbf{x}$ is

$$q(\mathbf{x}) = \mathbf{x}G\mathbf{x}^{\top}, \quad G = HH^{\top}.$$

An offset belongs to the search stencil exactly when

$$\min_{\mathbf{x} \in D_{\mathbf{k}}} \mathbf{x}G\mathbf{x}^{\top} \leq r_c^2.$$

Metric-tensor treatment of neighbor searching in parallelepiped cells has published precedent [3]. The exact minimization over every bin-offset box is the mdstats stencil construction rather than a transcription of that method.

Offsets failing this condition cannot contain a physical pair inside the cutoff.

9.3 Exact box minimization

Because G is positive definite, the objective is convex. At the constrained minimum, every coordinate is either:

- free;
- active at its lower bound;
- active at its upper bound.

There are only

$$3^3 = 27$$

active-set patterns. For every pattern, the free coordinates are obtained from the corresponding linear stationarity equations. Feasible points are evaluated and the smallest quadratic value is retained.

This procedure is exact up to floating-point linear algebra tolerance and does not use sampling.

9.4 Finite offset ranges

Before metric minimization, each axis uses the perpendicular-height lower bound to derive a finite integer range. The displacement interval must intersect

$$\left[-\frac{r_c}{h_\alpha}, \frac{r_c}{h_\alpha} \right].$$

Nonperiodic offset ranges are additionally limited by the finite number of bins.

Two hard limits protect against pathological geometries:

```
max_stencil_candidates
max_stencil_offsets
```

Exceeding either limit raises `CellListComplexityError`. S1 never silently truncates a stencil.

10 Candidate-bin traversal

Candidate atoms are stored in a mapping

```
bin index -> candidate-selection local slots
```

Local slots are stored in original candidate-selection order.

For each center atom:

1. determine its center bin;
2. add every retained stencil offset;
3. wrap periodic target-bin indices modulo the bin count;
4. reject out-of-range nonperiodic target bins;
5. collect candidate local slots from every visited occupied bin;
6. deduplicate local slots because multiple periodic offsets can map to the same physical bin when a periodic axis has few bins;
7. sort local slots to restore original candidate-selection order;
8. apply self-pair and pair-counting filters;
9. evaluate exact original-cell MIC geometry;
10. apply `distance < cutoff`;
11. append accepted records to the CSR row.

Pseudocode:

```
prepare search basis
transform and wrap search fractional coordinates
construct bins and exact metric stencil
assign candidate atoms to bins

for center in center_selection_order:
    slots = empty set
    for offset in stencil:
        target = center_bin + offset
        wrap periodic axes
        skip invalid nonperiodic target
        slots union= candidate_bin[target]

    candidates = candidate_selection[sorted(slots)]
    apply self/order filters
    vectors, distances, shifts = original_cell_MIC(candidates - center)
    accept distance < cutoff
    append CSR row
```


11 Original-basis final geometry

The search basis must not become the identity authority for periodic graph construction.

For every generated atom pair (i, j) , S1 recomputes

$$\mathbf{d}_{ij}^{\text{MIC}}, \quad r_{ij}, \quad \mathbf{m}_{ij}$$

using the original cell and the existing shared `minimum_image_geometry()` function.

Consequences:

- cell-list image shifts are directly comparable with dense shifts;
- lattice-reduction axis permutations do not leak into scientific output;
- mixed-PBC zero-shift requirements remain unchanged;
- exact cutoff behavior remains centralized;
- future graph modules need no cell-list-specific logic.

12 Implemented internal API

12.1 Backend selection

```
class NeighborSearchBackend(str, Enum):  
    DENSE = "dense"  
    CELL_LIST = "cell_list"
```

12.2 Cell-list options

```
@dataclass(frozen=True, slots=True)  
class CellListOptions:  
    use_lattice_reduction: bool = True  
    max_stencil_candidates: int = 1_000_000  
    max_stencil_offsets: int = 250_000  
    metric_tolerance: float = 1.0e-12  
    coordinate_tolerance: float = 1.0e-12  
    reduction_rtol: float = 1.0e-12  
    reduction_atol: float = 1.0e-12
```

All fields are validated and immutable.

12.3 Shared facade

```
def build_neighbor_list(  
    collection: AtomisticFrameCollection,  
    *,  
    frame_index: int,  
    center_indices: ArrayLike,  
    candidate_neighbor_indices: ArrayLike,  
    cutoff: float | PairCutoff,  
    pair_counting: PairCounting = PairCounting.DIRECTED,  
    backend: NeighborSearchBackend = NeighborSearchBackend.DENSE,  
    block_size: int = 256,  
    cell_list_options: CellListOptions | None = None,  
) -> NeighborListResult:  
    ...
```

`block_size` is used only by the dense backend. `cell_list_options` is used only by the cell-list backend.

`iter_neighbor_lists()` accepts the same backend and cell-list option fields but still performs an independent search for each frame. It is not a cache.

12.4 Internal plan and diagnostics

```
@dataclass(frozen=True, slots=True)
class CellListPlan:
    search_cell
    basis_transform
    inverse_basis_transform
    pbc
    bin_counts
    bin_origins
    bin_widths
    stencil_offsets
    reduction_applied

@dataclass(frozen=True, slots=True)
class CellListDiagnostics:
    reduction_applied: bool
    bin_counts: tuple[int, int, int]
    stencil_size: int
    occupied_candidate_bins: int
    bin_visits: int
    unique_candidate_pairs: int
    exact_pair_evaluations: int
    accepted_pairs: int
```

Diagnostics are developer-facing and are not part of scientific neighbor identity.

13 Complexity

Let:

- N_c be the number of centers;
- N_n be the number of candidate atoms;
- S be the stencil size;
- z_{bin} be the average number of unique candidates collected from visited bins.

Search-basis and stencil preparation are independent of atom-pair count. Atom assignment costs

$$O(N_c + N_n).$$

Bin traversal costs approximately

$$O(N_c S).$$

Exact pair geometry costs

$$O(N_c z_{\text{bin}}).$$

At fixed density, finite cutoff, and nonpathological bin geometry, S and z_{bin} remain bounded, giving approximately linear work:

$$O(N).$$

The worst case remains quadratic. Examples include:

- one bin containing nearly every atom;
- a cutoff near the system size;
- an extremely dense local cluster;
- a geometry that produces a very broad conservative stencil.

S1 makes no unconditional linear-time guarantee.

14 Exactness argument

No false negative is permitted.

For any true neighbor pair:

$$\|\Delta \mathbf{r}\| < r_c.$$

Its center and candidate atoms occupy some center bin and some periodic or nonperiodic candidate-bin image with offset $\mathbf{k}$. Their fractional displacement belongs to $D_{\mathbf{k}}$. Therefore,

$$\min_{\mathbf{x} \in D_{\mathbf{k}}} \mathbf{x} G \mathbf{x}^T \leq \|\Delta \mathbf{r}\|^2 < r_c^2.$$

Thus the metric test retains $\mathbf{k}$, traversal reaches the candidate bin, and the exact original-cell MIC calculation evaluates the pair.

The bin stage can produce false positives, but exact MIC and strict cutoff filtering remove them.

15 Error handling

S1 adds:

```
class CellListComplexityError(NeighborError): ...
```

The backend also uses existing errors:

- `InvalidNeighborSelectionError`;
- `InvalidNeighborCutoffError`;
- `UnsafeNeighborCutoffError`;
- `InvalidCellGeometryError`;
- `CoincidentAtomsError`.

The backend rejects:

- malformed or duplicate selections;
- partially overlapping selections;
- invalid pair-counting modes;
- nonfinite coordinates;
- singular cells;
- unsafe cutoffs;
- invalid lattice-reduction transformations;
- nonpositive bin widths or counts;
- stencil construction exceeding hard limits;
- distinct accepted atoms at effectively zero separation.

16 Determinism

Deterministic output is required.

- center rows follow the caller's center-selection order;
- candidate slots are restored to caller candidate-selection order;

- stencil offsets are sorted lexicographically;
- bin maps do not determine scientific row order;
- exact pair filtering uses the same shared MIC function as dense search;
- repeated runs must produce identical integer arrays and tolerance-identical floating arrays.

17 Dense-equivalence acceptance matrix

The implemented S1 tests cover:

1. cubic and orthorhombic periodic cells;
2. moderately skewed triclinic cells;
3. highly skewed nonsingular cells;
4. lattice reduction enabled and disabled;
5. fully periodic, two-dimensional periodic, one-dimensional periodic, and fully nonperiodic systems;
6. atoms crossing periodic boundaries;
7. directed disjoint selections;
8. unordered identical selections;
9. random candidate-selection permutations;
10. zero accepted pairs;
11. one accepted pair;
12. dense local clusters;
13. multiple chemical pair selections and cutoffs;
14. cutoffs near the conservative unique-image limit;
15. original-basis image-shift preservation after reduction;
16. deterministic repeated execution;
17. explicit cell-list iteration over selected frames;
18. option validation and hard complexity limits.

For every equivalence case:

```
cell-list pair identities == dense pair identities
cell-list image shifts   == dense image shifts
cell-list vectors        ~= dense vectors
cell-list distances      ~= dense distances
cell-list CSR grouping   == dense CSR grouping
```

18 Benchmark interpretation

The reproducible S1 benchmark is:

`benchmarks/cell_list_benchmark.py`

with outputs:

`benchmarks/cell_list_benchmark.json`

`benchmarks/cell_list_benchmark.md`

The benchmark records:

- system size and geometry;
- center and candidate counts;
- dense pair-evaluation count;
- cell-list exact-pair-evaluation count;
- candidate fraction;
- accepted-pair count;
- bin counts;
- stencil size;
- whether reduction was applied;
- dense and cell-list median runtime.

The timing data are machine-specific. They are used to characterize S1 and will inform, but not yet define, the later automatic crossover policy.

Small, chemically restricted searches can remain faster with the dense backend because cell-list plan construction has fixed overhead. S1 therefore exposes only explicit backend selection.

19 Interaction with consumer modules

RDF, coordination, bond-angle, and atomic-connectivity scientific algorithms do not manage cell bins or stencil geometry.

In S1, their existing calls continue to default to the dense backend. The cell-list backend can be exercised explicitly through the shared private neighbor facade and test harness.

Later integration must preserve the ownership rule:

```
scientific consumer specifies atoms, cutoff, and counting semantics
neighbor subsystem selects and executes the search mechanism
```

20 Deferred S2 contract

Stage S2 will build a Verlet candidate cache using a cell-list search radius

$$r_{\text{list}} = r_{\text{physical}} + r_{\text{skin}}.$$

S1 is intentionally stateless. `iter_neighbor_lists()` reconstructs the S1 plan for every frame. No S1 object may be interpreted as a valid trajectory cache.

S2 must continue to compare every cached frame against both:

- a fresh S1 cell-list result;
- a fresh dense result.

21 Reproduction checklist

An independent implementation reproduces S1 when it:

1. preserves the existing `NeighborListResult` contract;
2. uses the row-vector cell convention;
3. optionally transforms to an equivalent integer-reduced search basis;
4. bases bin dimensions on perpendicular cell-plane heights;
5. constructs a conservative metric-aware bin-offset stencil;
6. handles periodic wrapping and nonperiodic bounds explicitly;
7. deduplicates periodic bin aliases deterministically;
8. restores candidate-selection order before exact geometry evaluation;
9. computes final MIC vectors and image shifts in the original cell basis;
10. applies self, pair-counting, coincidence, and strict cutoff rules exactly as the dense backend;
11. raises rather than truncates pathological stencils;
12. passes the complete dense-equivalence matrix;
13. implements no hidden trajectory reuse.

22 References

1. Quentrec, B., and Brot, C. (1973). *New Method for Searching for Neighbors in Molecular Dynamics Computations*. Journal of Computational Physics, 13(3), 430-432. DOI: 10.1016/0021-9991(73)90046-6.

2. Heinz, T. N., and Hünenberger, P. H. (2004). *A Fast Pairlist-Construction Algorithm for Molecular Simulations under Periodic Boundary Conditions*. Journal of Computational Chemistry, 25(12), 1474-1486. DOI: 10.1002/jcc.20071.
3. Cui, Z., Sun, Y., and Qu, J. (2009). *The Neighbor List Algorithm for a Parallelepiped Box in Molecular Dynamics Simulations*. Chinese Science Bulletin, 54(9), 1463-1469. DOI: 10.1007/s11434-009-0197-0.
4. Rogers, D. M. (2016). *Overcoming the Minimum Image Constraint Using the Closest Point Search*. Journal of Molecular Graphics and Modelling, 68, 197-205. DOI: 10.1016/j.jmgm.2016.07.004.
5. Larsen, A. H., et al. (2017). *The Atomic Simulation Environment - A Python Library for Working with Atoms*. Journal of Physics: Condensed Matter, 29(27), 273002. DOI: 10.1088/1361-648X/aa680e.
6. Nguyen, P. Q., and Stehlé, D. (2009). *Low-Dimensional Lattice Basis Reduction Revisited*. ACM Transactions on Algorithms, 5(4), Article 46. DOI: 10.1145/1597036.1597050.